

Docker Compose CheatSheet - Modul 347

Inhaltsverzeichnis

- Docker Compose CheatSheet - Modul 347
 -  Inhaltsverzeichnis
 -  YAML Grundlagen
 - YAML vs JSON
 - Arrays und Objekte
 - Variablen und Referenzen
 -  Docker Compose Struktur
 - Grundlegende compose.yaml
 - Services Definition
 -  Service-Konfiguration
 - Basis-Optionen
 - Umgebungsvariablen
 - Volumes
 - Netzwerk & Ports
 - Abhängigkeiten
 -  Docker Compose Befehle
 - Starten & Stoppen
 - Build & Management
 - Debugging
 -  Praktische Beispiele
 - Einfacher Web-Stack
 - Datenbank mit phpMyAdmin
 - WordPress Stack
 - Flask + Redis + Nginx
 - PostgreSQL + pgAdmin
 -  Profile & Skalierung
 - Service-Profile
 - Skalierung
 -  Sicherheit & Best Practices
 - .env Dateien
 - Secrets
 - Best Practices
 -  Typische Workflows
 - Entwicklung
 - Produktion
 - Debugging
 -  Wichtige Hinweise
 - Volume-Management
 - Netzwerk
 - Restart-Policies

YAML Grundlagen

YAML vs JSON

JSON:

```
{
  "key": "Wert",
  "key2": "asdfgg\nTest"
}
```

YAML:

```
key: Wert
key2: "asdfgg\nTest"
```

Arrays und Objekte

Einfaches Array:

```
themen:
- Syntax
- Listen
- Objekte
- Referenzen
```

Array mit Objekten:

```
personen:
- name: Anna
  rolle: Lehrerin
- name: Tom
  rolle: Lernender
```

Variablen und Referenzen

```
name: &lernender Max Mustermann
best_lernender: *lernender

hauptperson: &haupt Max Mustermann
beste_person: *haupt
```

Docker Compose Struktur

Grundlegende compose.yaml

```
services:
  service-name:
    image: image-name
    # oder
    build: ./path

volumes:
  volume-name:

networks:
  network-name:
```

Wichtig: `version: '3'` ist veraltet und nicht mehr erforderlich!

Services Definition

```
services:
  web:
    image: nginx
    build: ./app
    container_name: my_web
    ports:
      - "8080:80"
    volumes:
      - ./data:/app/data
    environment:
      - ENV_VAR=value
    depends_on:
      - db
    restart: always
```

Service-Konfiguration

Basis-Optionen

```
services:
  app:
    image: nginx           # Verwende Image
    build: ./path          # Baue aus Dockerfile
    container_name: my_container # Container-Name setzen
    command: ["nginx", "-g", "daemon off;"] # Command überschreiben
```

Umgebungsvariablen

```
services:
  app:
    environment:
      - MYSQL_ROOT_PASSWORD=password
      - MYSQL_DATABASE=mydb
      - DEBUG=true
    # oder
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: mydb
```

Mit .env-Datei:

```
services:
  db:
    environment:
      - MYSQL_ROOT_PASSWORD=${DB_PASSWORD}
```

Volumes**Named Volumes:**

```
services:
  db:
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
    name: custom_db_name
```

Bind Mounts:

```
services:
  web:
    volumes:
      - ./website:/usr/share/nginx/html:ro
      - ./config:/etc/nginx/conf.d
```

Netzwerk & Ports

```
services:
  web:
    ports:
      - "8080:80"          # Host:Container
      - "443:443"          # Nur Container-Port (auto-assign Host)
      - "80"               # Nur für andere Container
    expose:
      - "3000"             # Nur für andere Container
```

Abhängigkeiten

```
services:
  web:
    depends_on:
      - db
      - redis
    restart: always          # always, no, on-failure, unless-stopped
    restart: on-failure:10   # Max 10 Versuche
```

Docker Compose Befehle

Starten & Stoppen

```
# Standard starten
docker compose up

# Im Hintergrund starten
docker compose up -d

# Mit bestimmten Profilen
docker compose --profile frontend up

# Stoppen
docker compose down

# Stoppen mit Volume-Löschung
docker compose down -v

# Pause/Unpause
docker compose pause
docker compose unpause
```

Build & Management

```
# Bauen (bei Dockerfile-Änderungen)
docker compose up --build

# Nur bauen ohne starten
docker compose build

# Einzelnen Service bauen
docker compose build web

# Services skalieren
docker compose up --scale web=3

# Logs anzeigen
docker compose logs
docker compose logs web
```

Debugging

```
# Laufende Services anzeigen
docker compose ps

# In Container einsteigen
docker compose exec web bash

# Kommando in Service ausführen
docker compose run web ls -la

# Container neu starten
docker compose restart web
```

Praktische Beispiele

Einfacher Web-Stack

```
services:
  web:
    image: nginx
    ports:
      - "7080:80"

  web2:
    build: ./httpd
    ports:
      - "7081:80"
```

Datenbank mit phpMyAdmin

```
services:
  db:
    image: mariadb
    environment:
      - MARIADB_ROOT_PASSWORD=sml12345
      - MARIADB_DATABASE=blog
      - MARIADB_USER=blog
      - MARIADB_PASSWORD=sml12345
    restart: always
    volumes:
      - db_data:/var/lib/mysql

  pma:
    image: phpmyadmin
    environment:
      - PMA_HOST=db
    ports:
      - "6080:80"
    restart: always
    depends_on:
      - db

volumes:
  db_data:
    name: db_data_wordpress
```

WordPress Stack

```
services:
  db:
    image: mariadb
    environment:
      - MYSQL_ROOT_PASSWORD=sml12345
      - MARIADB_DATABASE=blog
      - MARIADB_USER=blog
      - MARIADB_PASSWORD=sml12345
    restart: on-failure:10
    volumes:
      - db_data:/var/lib/mysql
    profiles:
      - backend

  wp:
    image: wordpress
    environment:
      - WORDPRESS_DB_HOST=db
      - WORDPRESS_DB_USER=blog
      - WORDPRESS_DB_PASSWORD=sml12345
      - WORDPRESS_DB_NAME=blog
    ports:
      - "6081:80"
    restart: on-failure:10
    depends_on:
      - db
    volumes:
      - wp_data:/var/www/html
    profiles:
      - frontend

  pma:
    image: phpmyadmin
    environment:
      - PMA_HOST=db
    ports:
      - "6080:80"
    restart: on-failure:10
    depends_on:
      - db
    profiles:
      - debug

volumes:
  db_data:
    name: db_data_wordpress
  wp_data:
    name: wp_data_wordpress
```

Flask + Redis + Nginx

```
services:
  flask:
    build: .
    container_name: flask_app
    depends_on:
      - redis
    environment:
      - REDIS_HOST=redis

  redis:
    image: redis:alpine
    container_name: redis_cache

  nginx:
    image: nginx
    ports:
      - "3333:80"
    depends_on:
      - flask
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
```

PostgreSQL + pgAdmin

```
services:
  postgres:
    image: postgres
    environment:
      - POSTGRES_DB=uebungsdatenbank
      - POSTGRES_PASSWORD=password
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d

  pgadmin:
    image: dpage/pgadmin4
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@example.com
      - PGADMIN_DEFAULT_PASSWORD=admin
    ports:
      - "5050:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    depends_on:
      - postgres

volumes:
  postgres_data:
  pgadmin_data:
```


Profile & Skalierung

Service-Profile

```
services:
  db:
    image: mariadb
    profiles:
      - backend

  web:
    image: nginx
    profiles:
      - frontend

  debug:
    image: phpmyadmin
    profiles:
      - debug
```

Verwendung:

```
# Nur Frontend
docker compose --profile frontend up

# Mehrere Profile
docker compose --profile frontend --profile debug up

# Alle Services ohne Profile + bestimmte Profile
docker compose --profile debug up
```

Skalierung

```
services:
  web:
    image: nginx
    ports:
      - "80" # Automatische Port-Zuweisung
    deploy:
      replicas: 3
```

Manuell skalieren:

```
docker compose up --scale web=5
```

Sicherheit & Best Practices

.env Dateien

.env:

```
DB_PASSWORD=sm112345
MYSQL_ROOT_PASSWORD=supersecret
API_KEY=your-secret-key
```

compose.yaml:

```
services:
  db:
    environment:
      - MYSQL_ROOT_PASSWORD=${DB_PASSWORD}
      - API_KEY=${API_KEY}
```

Secrets

```
services:
  app:
    secrets:
      - db_password
      - api_key

secrets:
  db_password:
    file: ./secrets/db_password.txt
  api_key:
    external: true
```

Best Practices

1. **Verwenden Sie .env für sensible Daten**
2. **Named Volumes für Datenpersistenz**
3. **depends_on für Service-Reihenfolge**
4. **restart: on-failure für Produktionsumgebungen**
5. **Profile für verschiedene Umgebungen**
6. **Bind Mounts nur für Entwicklung**
7. **Spezifische Image-Tags (nicht latest)**

 Typische Workflows**Entwicklung**

```
# Mit Build starten
docker compose up --build

# Logs verfolgen
docker compose logs -f

# Service neu starten
docker compose restart web
```

Produktion

```
# Im Hintergrund starten
docker compose up -d

# Status prüfen
docker compose ps

# Updates
docker compose pull
docker compose up -d
```

Debugging

```
# Container-Details
docker compose ps
docker compose logs service-name

# In Container einsteigen
docker compose exec service-name bash

# Neue Container-Instanz für Debugging
docker compose run service-name bash
```

⚠ Wichtige Hinweise

Volume-Management

- **Named Volumes** überleben `docker compose down`
- **Bind Mounts** spiegeln lokale Dateien direkt
- Bei DB-Passwort-Änderungen: Volume neu erstellen!

Netzwerk

- Services können sich über Service-Namen erreichen
- `depends_on` garantiert nur Start-Reihenfolge, nicht Verfügbarkeit
- `links` ist deprecated, verwenden Sie Service-Namen

Restart-Policies

- `no`: Niemals neu starten
- `always`: Immer neu starten
- `on-failure`: Nur bei Fehlern
- `unless-stopped`: Außer manuell gestoppt

Erstellt für Modul 347 - Docker Compose Prüfungsvorbereitung